



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE INGENIERÍA
Año 2024 – 2.^{do} Cuatrimestre

REDES NEURONALES

Departamento de Electrónica
Trabajo Práctico II

2 de Noviembre de 2024

Autor	Padrón	Correo electrónico
Gómez Villafañe, Marcos	— # 105 055 —	mgomezv@fi.uba.ar

Índice

1. Introducción	3
2. Perceptrón Simple AND y OR	3
2.1. Función lógica AND	4
2.1.1. Error y frontera de decisión	4
2.1.2. AND de 4 entradas	4
2.2. Función lógica OR	5
2.2.1. Error y frontera de decision	5
2.2.2. OR de 4 entradas	6
2.3. Capacidad	7
3. Perceptrón Multicapa	8
3.1. XOR de 2 entradas	9
3.2. XOR de 4 entradas	9
3.3. Función continua	10
3.3.1. Diferentes Batchsizes	13
4. Máquina restringida de Boltzmann	13
5. Red convolucional	17
5.1. Comparación con perceptrón multicapa	19
6. Autoencoder	20
7. Conclusión	24
8. Apéndice	25

1. Introducción

En este trabajo práctico se entrenan perceptrones simples y perceptrones multicapas para aprender funciones lógicas AND, OR y XOR. Adicionalmente, se analizan las propiedades del perceptrón como su capacidad y su evolución con el error. Por otro lado, se desarrolla una red neuronal multicapa que se entrena con backpropagation. Finalmente, se analiza e implementa una maquina de Boltzmann, una red convolucional y un autoencoder con imágenes de la base de datos MNIST.

2. Perceptrón Simple AND y OR

El perceptrón simple consta de una única neurona con pesos sinápticos w_i , $1 \leq i \leq D$ conectando a cada dimensión i del vector de entrada $\vec{x} \in \mathcal{R}^D$ a la entrada. El perceptron simple se muestra en la Figura 2.1.

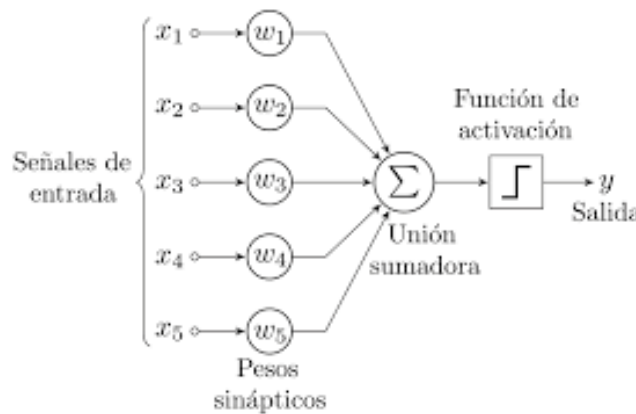


Figura 2.1: Representación del perceptrón simple

Notar que la unión sumadora y la función de activación (función signo) son las mismas que las de una neurona de la red de Hopfield. La salida y corresponde entonces a

$$\hat{y} = \text{Sign}(\vec{w}^T \vec{x}) \quad (2.1)$$

donde la salida del perceptrón $\hat{y}$ es la predicción de la red neuronal sobre la clase a la que pertenece el vector de entrada. El perceptrón simple aprenderá las funciones lógicas AND y OR, con vectores de entrada de dimensión 2. Luego, se entrenaran perceptrones con funciones AND y OR de 4 entradas. Se menciona sin embargo, que toda función lógica de una cantidad arbitraria de términos puede descomponerse en funciones lógicas de dos variables, por lo cual un perceptron que resuelve AND y OR de 2 variables puede resolver AND y OR de 4 variables también (bajo unas modificaciones de código).

2.1. Función lógica AND

Dada la tabla de la función lógica AND para dos variables x_1, x_2 que toman valores 1 para verdadero y -1 para falso: con lo cual se tienen 4 vectores de dos dimensiones y 4 salidas y .

x_1	x_2	$y = x_1 \wedge x_2$
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

Tabla 1: Tabla de Verdad de AND ($\wedge$)

2.1.1. Error y frontera de decisión

En la Figura 2.4 se muestra la evolución del error del perceptrón AND y la recta de decisión para el perceptrón AND de 2 entradas. El perceptron converge relativamente rápido, en solo 8 iteraciones

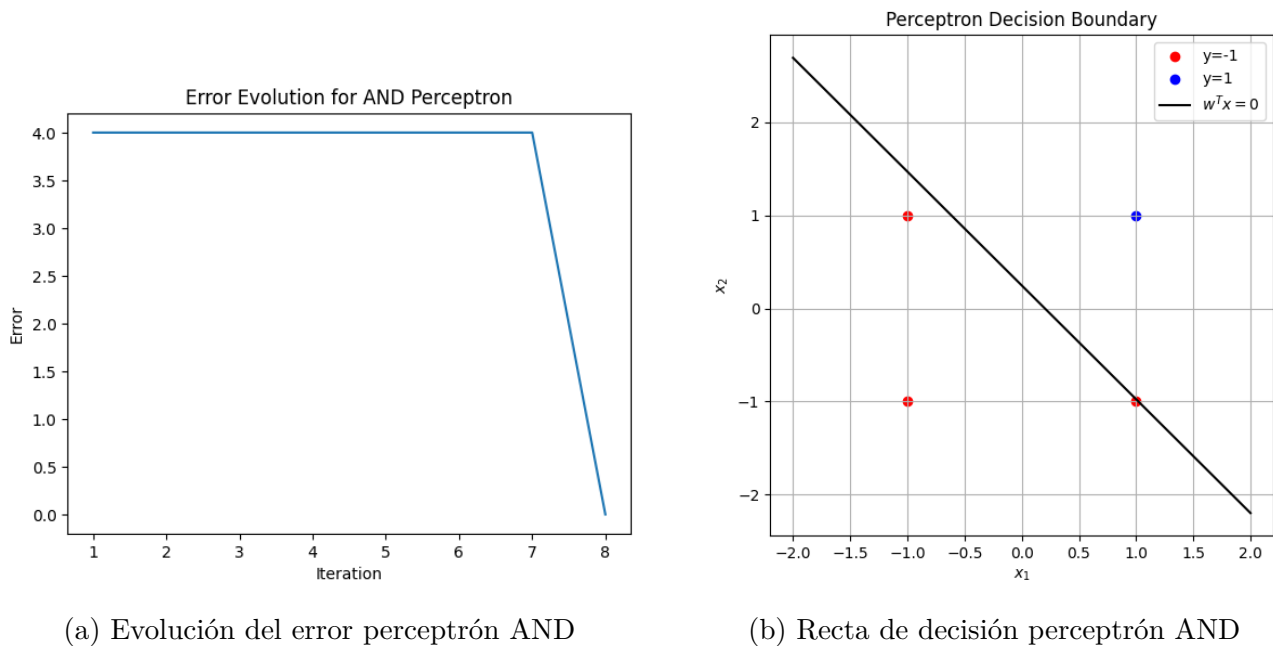


Figura 2.2: Gráficos perceptron AND de 2 entradas

2.1.2. AND de 4 entradas

La tabla de la AND de 2 entradas se amplia para 4 entradas. Se vuelve a entrenar el perceptron para aprender la función AND de 4 entradas. La Figura 2.3 muestra la evolución del error del perceptron

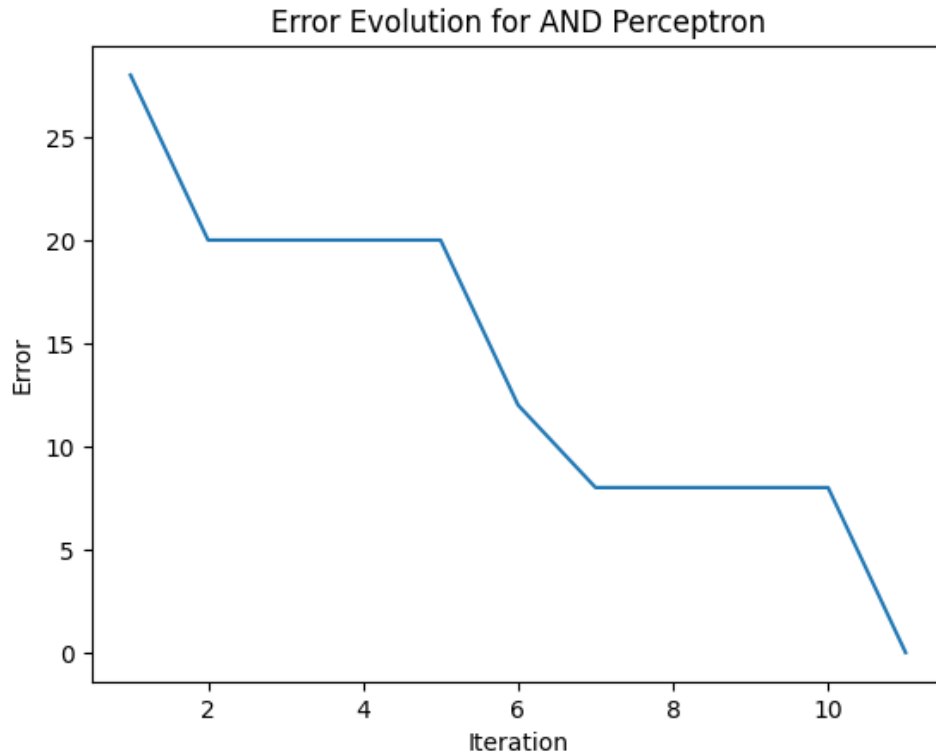


Figura 2.3: Evolución del error del perceptron OR de 4 entradas

se nota una evolución mas escalonada para la AND de 4 entradas, esto se debe a que hay una cantidad de posibilidades mayor. Converge tras 11 iteraciones.

2.2. Función lógica OR

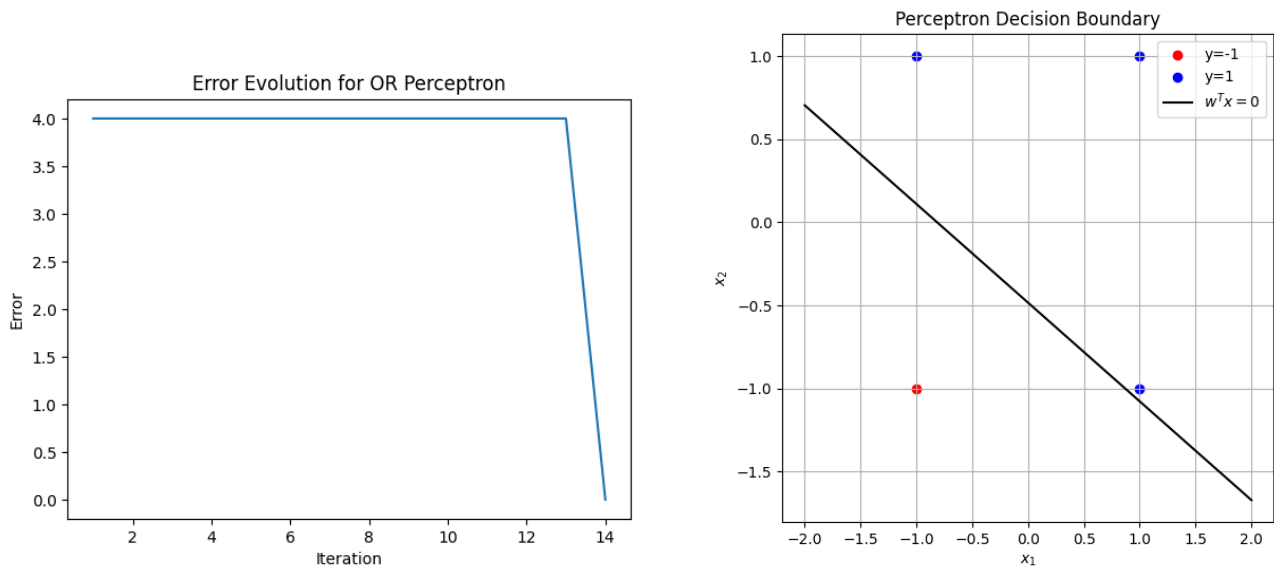
Dada la tabla de la función lógica OR para dos variables x_1, x_2 que toman valores 1 para verdadero y -1 para falso: con lo cual se tienen 4 vectores de dos dimensiones y 4 salidas y .

x_1	x_2	$y = x_1 \vee x_2$
-1	-1	-1
-1	1	1
1	-1	1
1	1	1

Tabla 2: Tabla de Verdad de OR ($\vee$)

2.2.1. Error y frontera de decision

En la Figura 2.4 se muestra la evolución del error del perceptrón OR y la recta de decisión para el perceptrón OR de 2 entradas. El perceptron converge relativamente rapido, en solo 14 iteraciones



(a) Evolución del error perceptrón OR

(b) Recta de decisión perceptrón OR

Figura 2.4: Gráficos perceptron OR de 2 entradas

2.2.2. OR de 4 entradas

La tabla de la OR de 2 entradas se amplía para 4 entradas. Se vuelve a entrenar el perceptron para aprender la función OR de 4 entradas. La Figura 2.5 muestra la evolución del error del perceptron

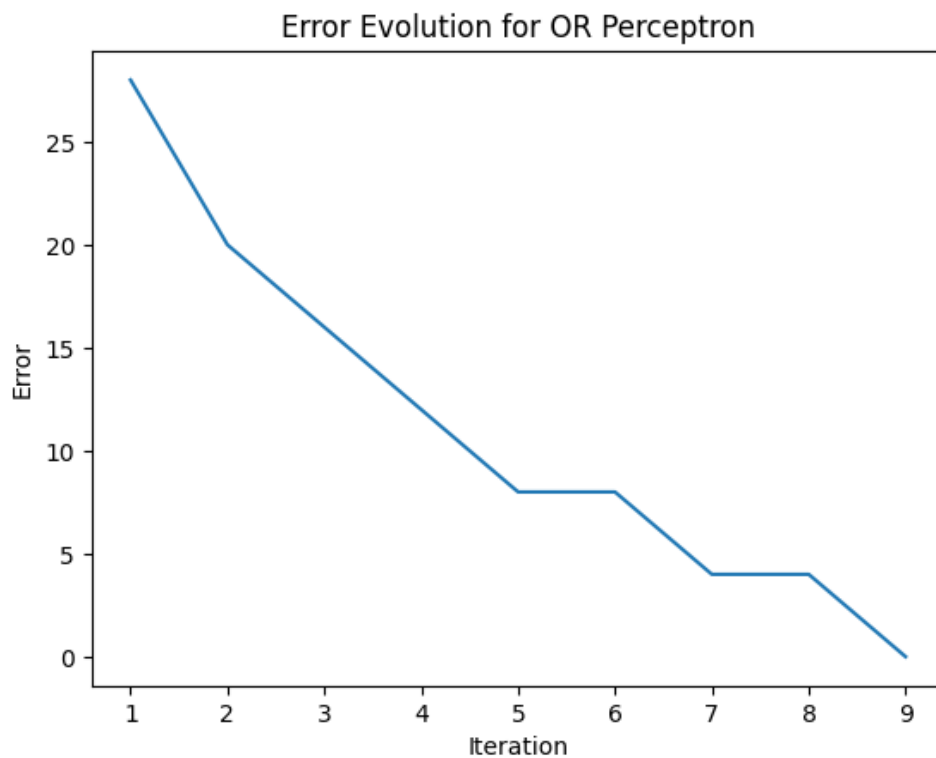


Figura 2.5: Evolución del error del perceptron OR de 4 entradas

se nota una evolución mas escalonada para la OR de 4 entradas, esto se debe a que hay una cantidad de posibilidades mayor. Converge tras 9 iteraciones.

2.3. Capacidad

Para medir la capacidad de un perceptrón, se lo entrena con una cantidad ascendente de patrones a aprender y se mide la proporción de los patrones aprendidos contra la cantidad total de patrones. Esto mide la capacidad del perceptrón. Para una dimensionalidad de 15 (patrones de 15 dimensiones), la Figura 2.6 muestra la capacidad del perceptrón

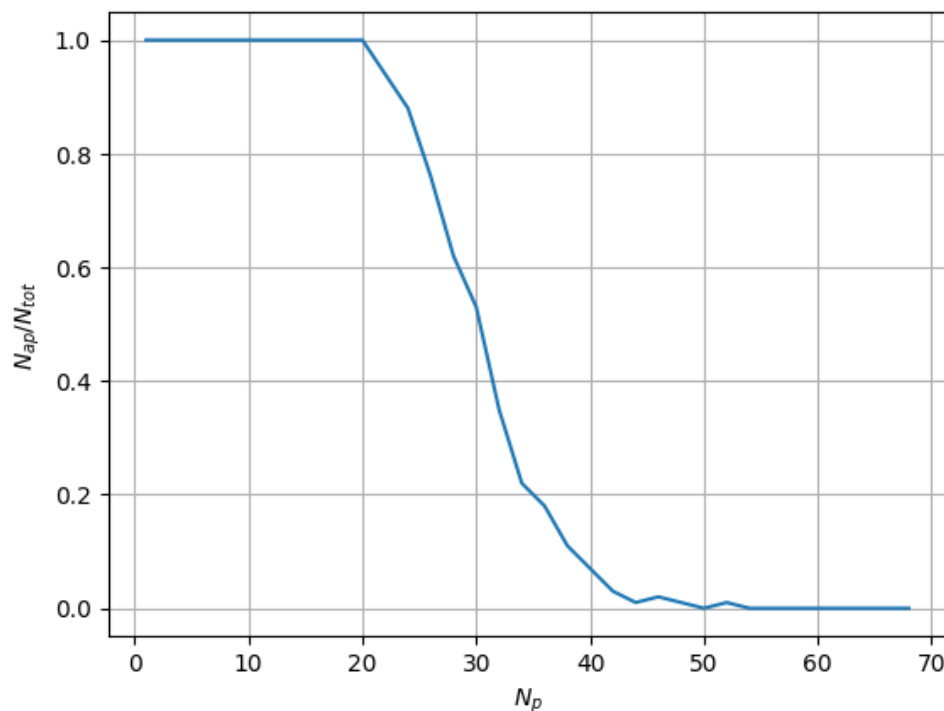


Figura 2.6: Capacidad del perceptrón para dimensión 15

Se observa que para una cantidad de patrones a aprender de 2 veces la dimensionalidad (30), el perceptrón solo aprende la mitad de los patrones.

Se comparan las capacidades del perceptrón para 2 dimensionalidades en la Figura ??

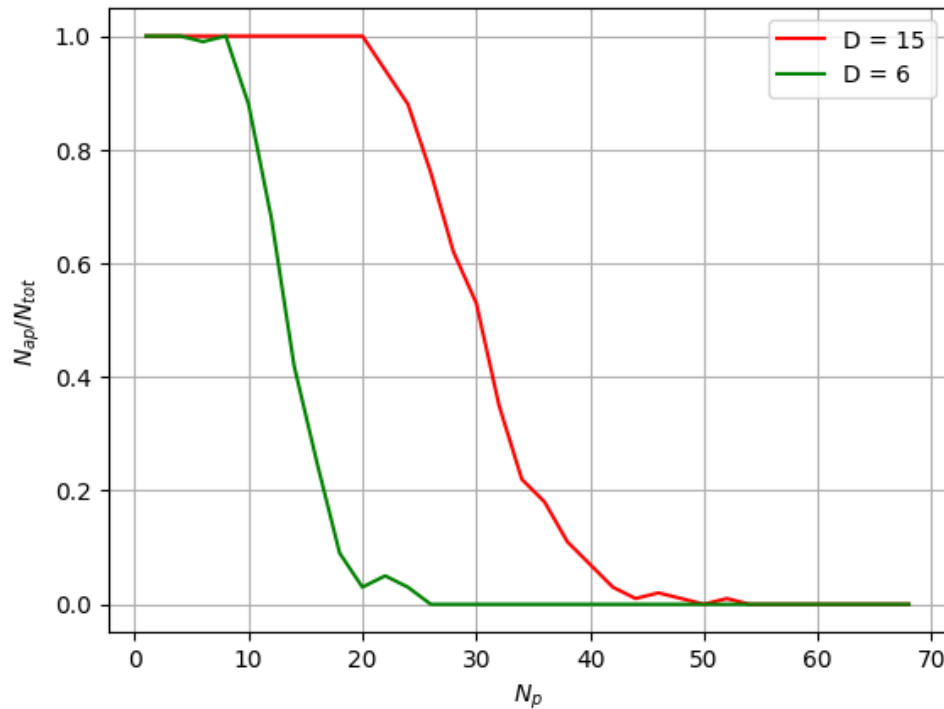


Figura 2.7: Capacidad del perceptrón para dimensión 6

La pendiente en el de menor dimensionalidad es mayor que la pendiente en el de mayor dimensionalidad y se sigue cumpliendo que en aproximadamente $2N$, la proporción de patrones aprendidos es 50 %.

3. Perceptrón Multicapa

El Perceptrón Multicapa consta de una capa de neuronas de entrada, una capa de salida, y capas ocultas entre las dos. Para una capa i y una capa subsecuente j , cada neurona de la capa i se conecta a todas las neuronas de la capa j . Esto se aplica para todas las capas de la red neuronal.

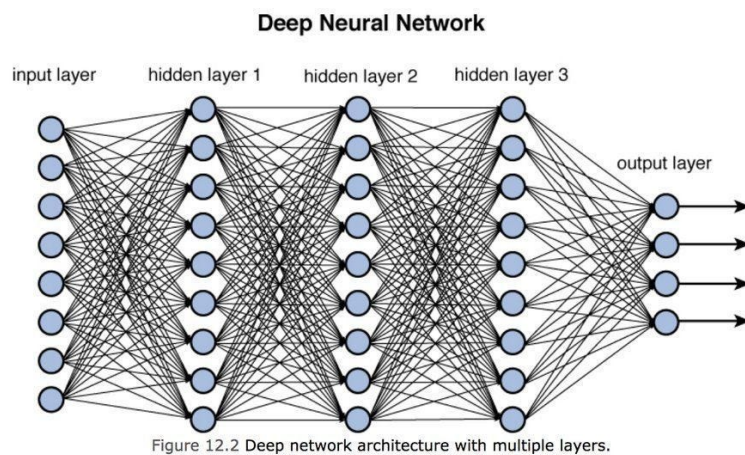


Figura 3.1: Perceptrón multicapa

El método para entrenar este tipo de redes neuronales se llama backpropagation. Consta de buscar la derivada parcial del error de la salida con respecto a cada peso $\frac{\partial E}{\partial w_{ij}}$ y actualizar dichos pesos mediante

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} \quad (3.1)$$

Donde η es la constante (tasa) de aprendizaje.

3.1. XOR de 2 entradas

Para la XOR de 2 entradas se opta por un perceptrón multicapa simple que consta de una sola capa oculta con 4 neuronas. Por lo tanto, la red neuronal es: 2 neuronas de entrada, 4 neuronas de la capa oculta, y 1 neurona de salida. La constante de aprendizaje tiene el valor $\eta = 0,01$, el error mínimo solicitado es $e_{min} = 0,01$ y la cantidad máxima de iteraciones es 100000.

La Figura 3.2 muestra la evolución del error cuadrático medio con las iteraciones

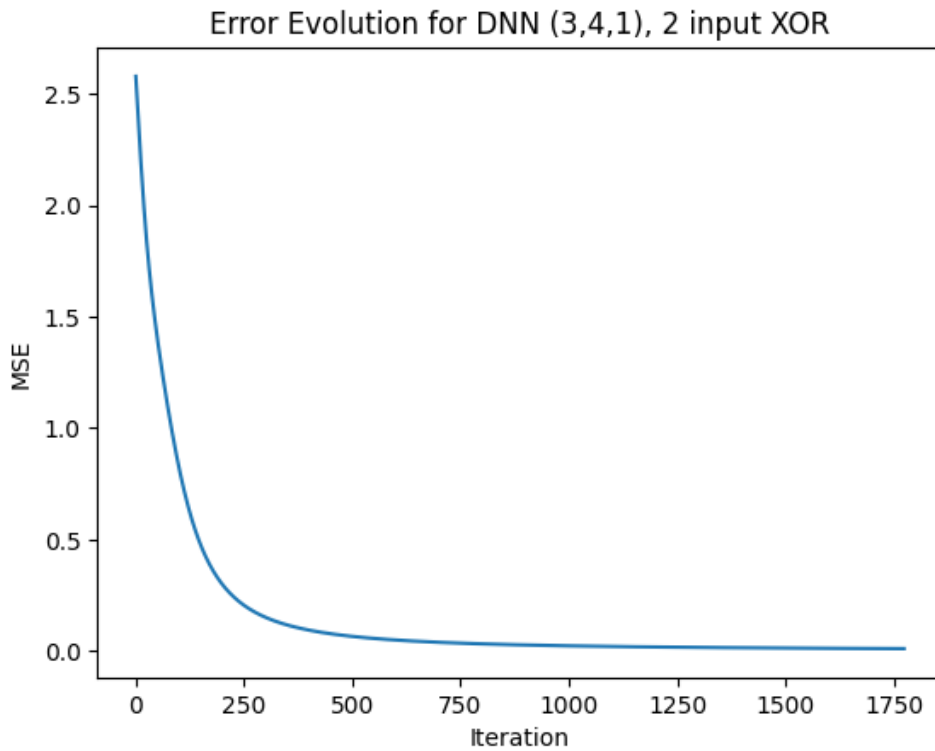


Figura 3.2: XOR de 2 entradas, evolución del error

Alrededor de la iteración 1750, la red neuronal converge a un error cuadrático medio menor a 0.01.

3.2. XOR de 4 entradas

Dado que ahora se trabaja con un conjunto de datos de mayor magnitud, se utiliza una red neuronal con una sola capa oculta compuesta por 16 neuronas. Por lo tanto, la red neuronal

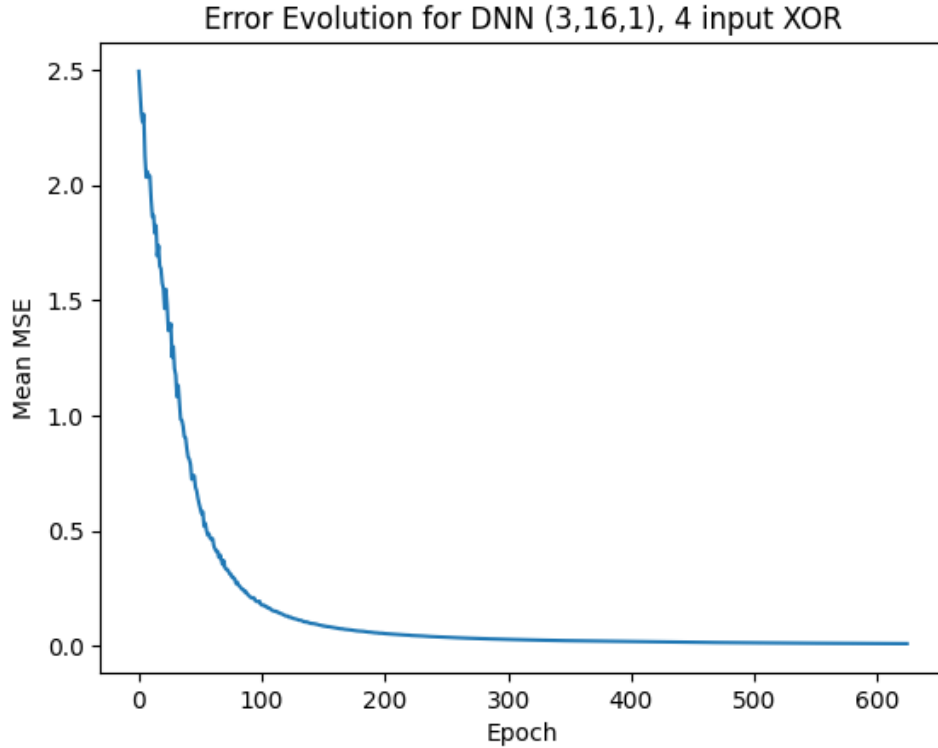


Figura 3.3: xor4 entradas

consta de 4 neuronas de entrada, 16 neuronas intermedias y 11 neurona de salida

Para la XOR de 4 entradas, se entrena la red neuronal con backpropagation mediante batches, donde cada batch es un subconjunto del dataset de entrenamiento y el batchsize es el tamaño de dicho batch. Esto quiere decir, que si el batchsize es 5 y el dataset tiene 100 muestras, se seleccionaran 5 muestras sin reposición y de forma aleatoria, se hará la actualización de los pesos con dichas muestras y luego se seleccionaran otras 5 muestras de las 95 muestras restantes y se repite el proceso. Este proceso se repite hasta que no hay mas muestras para seleccionar, finalizando un epoch de entrenamiento. Se usa este método de batches para que el método del gradiente descendiente no se estanque en mínimos locales tan fácilmente.

El batchsize utilizado es en 4, el learning rate es $\eta = 0,01$, el erro mínimo es 0.01 y la máxima cantidad de epochs es 10000. La figura 3.3 muestra la evolucion del error con las epochs.

La convergencia aparenta ser mas rápida que para la XOR de 2 entradas. Esto se debe a que se utilizo el método de batch update, donde realmente para cada epoch, se hicieron $16/4=4$ actualizaciones, con lo cual la cantidad de actualizaciones totales para la XOR de 4 entradas es mayor que para la XOR de 2 entradas.

3.3. Función continua

Dada la siguiente función continua

$$f(x, y, z) = \sin(x) + \cos(y) + z \quad ; \quad x \in [0, 2\pi] , y \in [0, 2\pi] , z \in [-1, 1] \quad (3.2)$$

Se propone entrenar un perceptrón multicapa que aprenda la función continua. Para ello, se hace uso del teorema postulado por Gulyev y Ismailov ¹, donde cualquier función continua multivariable puede aproximarse con 2 capas ocultas. Por lo tanto, se elige usar un perceptrón multicapa con 2 capas ocultas de 16 neuronas cada una, 1 capa de entrada con 3 neuronas y una capa de salida con 1 neurona.

Para nuestro entrenamiento, se tienen 1000 puntos para entrenar y se elige un batchsize de 10. El límite de epochs es 1000 y el error mínimo a alcanzar es 0.01 (error cuadrático medio). El η elegido para el entrenamiento es 0.01. Los pesos se inicializan de la misma forma que antes.

La Figura 3.4 muestra la evolución del error con los epochs

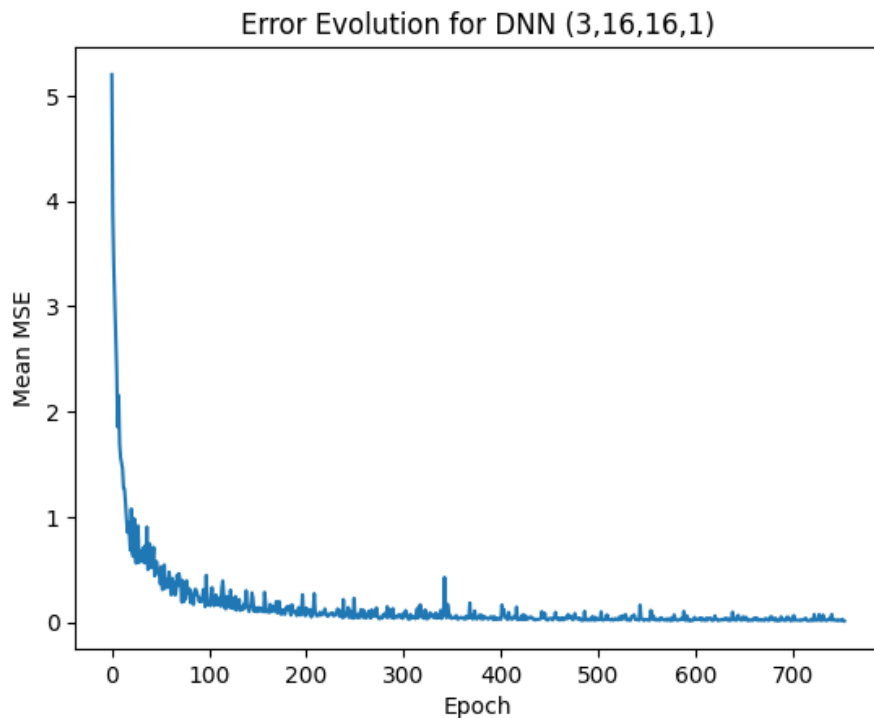


Figura 3.4: Evolución del error perceptrón multicapa función continua

Se observa que el error mínimo se alcanza previo a los 1000 epochs, alrededor del epoch 800.

¹<https://arxiv.org/abs/2101.09181>

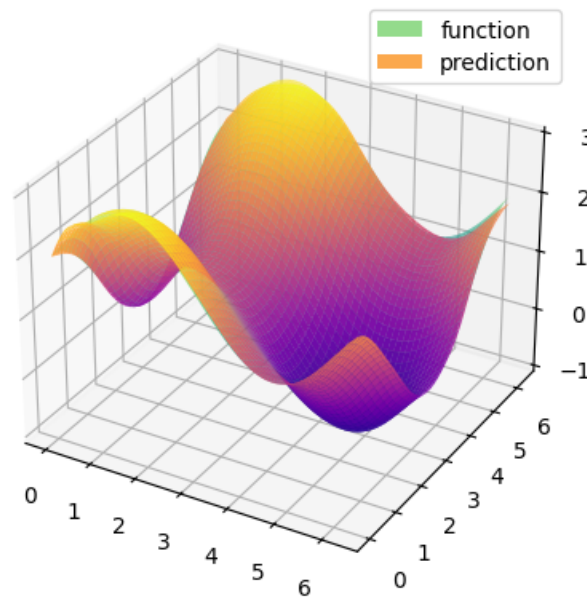


Figura 3.5: Aproximación de la red neuronal y función $f(x, y, z)$ superpuestas. Se fija $Z=1$ para visualizarlo en 3D.

La red neuronal logra aproximar la función de forma bastante acertada, incluso para puntos no pertenecientes al dataset de entrenamiento, indicando un buen aprendizaje. Se reduce una dimension mas, fijando $y = 2\pi$ y se obtiene la Figura 3.6

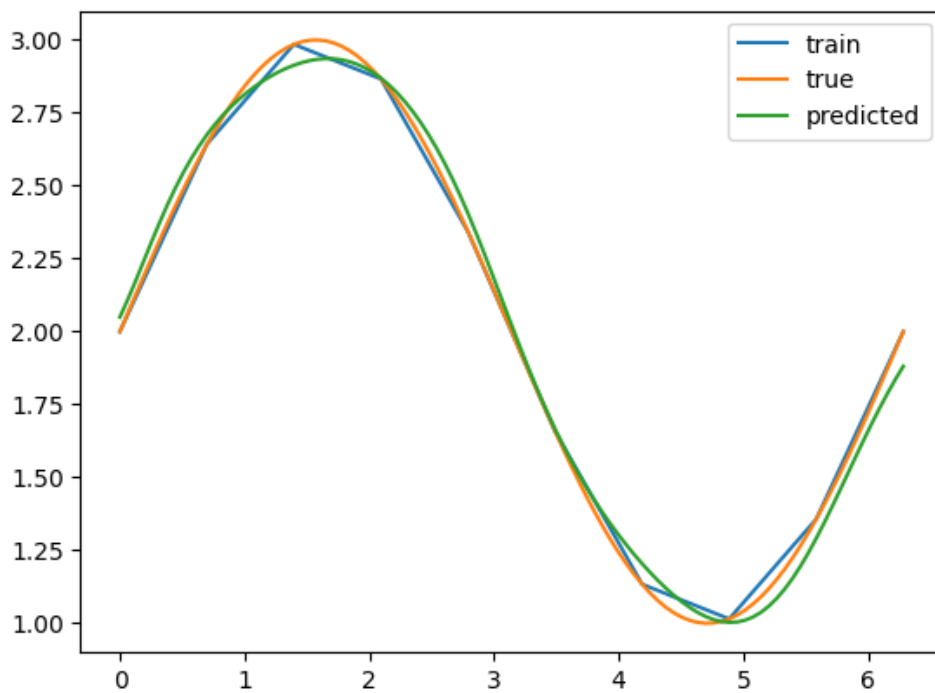
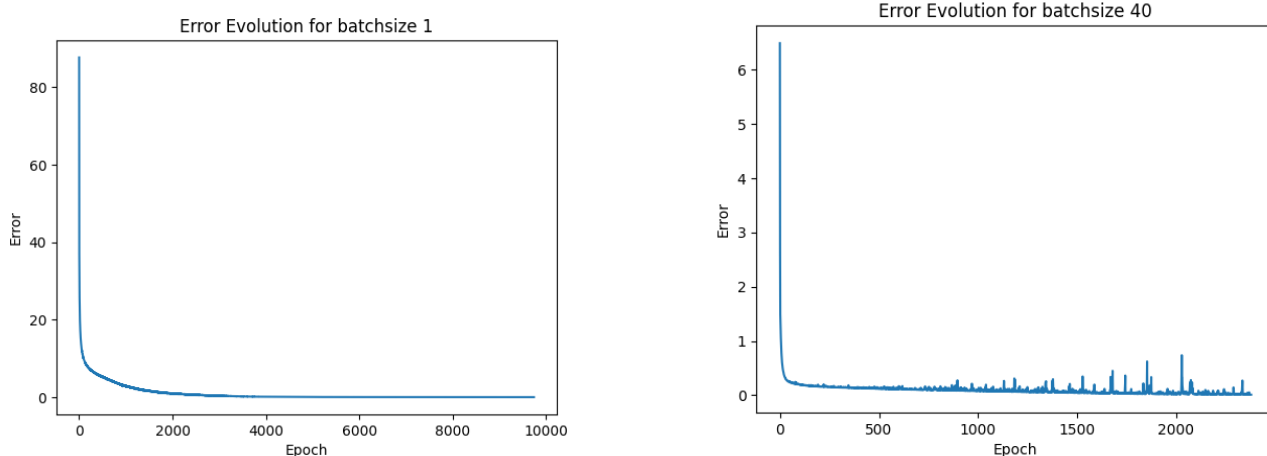


Figura 3.6: Puntos de entrenamiento, predicción y función real

3.3.1. Diferentes Batchesizes

En esta sección se usan batchsizes de diferentes tamaños para entrenar un perceptron multicapa de una única capa oculta con 30 neuronas. La tasa de aprendizaje se fija en 0.001 y se entrena una vez con un batchsize de 1 y una segunda vez con un batchsize de 40. La Figura 3.7 muestra los resultados del entrenamiento.



(a) Evolución del error perceptrón una capa con batchsize 1

(b) Evolución del error perceptrón una capa con batchsize 40

Figura 3.7: Gráficos Error para diferentes Batchesizes

Se observa que entrenar con un batchsize de 1 tiene una convergencia menos ruidosa al error final, mientras que en el batchsize de 40 la convergencia es mas ruidosa, con picos repentinos de error. Esto se contradice con el pensamiento inicial de que un batchsize de mayor tamaño da lugar a una convergencia mas suave dado que el promediado del error disminuye lentamente. Se esperaba que el batchsize mas chico ocasione picos de error debido a la actualización de una muestra a la vez. La razón por la cual se ve suave es porque el error mostrado es la suma de los errores de las actualizaciones dividido el batchsize. Con lo cual, en realidad, el error de cada actualización de los pesos es mas ruidoso.

En cuanto a tiempos de convergencia, se observa que un batchsize muy chico hace que le tome mas tiempo a la red neuronal converger. Si bien realiza muchas mas actualizaciones con un batchsize chico, estas actualizaciones pueden perjudicar a la mayoría de las muestras con el fin de mejorar el resultado de una única muestra, perdiendo en parte la memoria. Es por esto que probablemente le lleve mas tiempo converger con un batchsize de 1.

4. Máquina restringida de Boltzmann

La Figura 4.1 muestra un esquema de la máquina restringida de Boltzmann

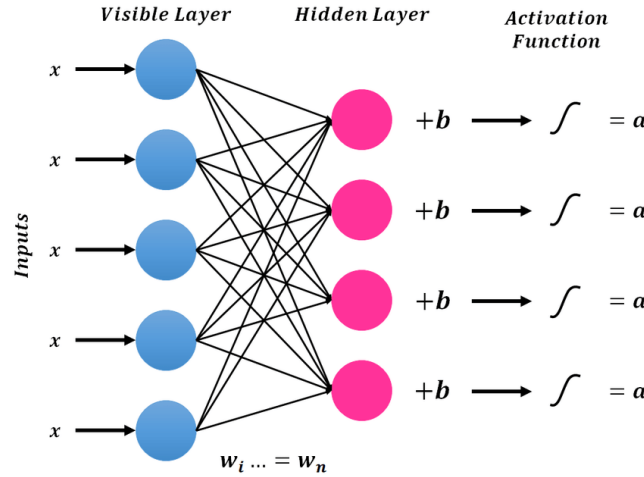


Figura 4.1: Máquina Restringida de Boltzmann

El procedimiento para entrenar este tipo de red neuronal consta de 2 fases principales, primero introducir la imagen en la capa visible (azul, $v_{1,data}$), realizar un pase hacia adelante a la capa oculta, usar las activaciones para predecir los valores de la capa oculta ($h_{1,data}$), usar los valores de la capa oculta para activar hacia atrás la capa visible ($v_{1,recon}$) y luego volver a estimar los valores de la capa oculta ($h_{1,recon}$). El error para la actualización de pesos se calcula de la siguiente forma:

$$\begin{aligned}\Delta W_{ij} &= \eta (\langle V_i h_j \rangle_{data} - \langle V_i h_j \rangle_{recon}) \\ \Delta b_i &= \eta (\langle V_i \rangle_{data} - \langle V_i \rangle_{recon}) \\ \Delta b_j &= \eta (\langle h_j \rangle_{data} - \langle h_j \rangle_{recon})\end{aligned}$$

Los corchetes $\langle, \rangle$ representan el promedio sobre las muestras. Se opta por una Máquina Restringida de Boltzmann de 784 neuroneas en la capa visible y 100 en la oculta.

Se analizaron 3 métodos de aprendizaje:

1. Con sampleo de normal gaussiana para la entrada y la reconstrucción
2. Con sampleo de normal gaussiana unicamente para la entrada
3. Sin sampleo

De los 3 métodos que se analizaron, el sampleo con normal gaussiana tanto en la entrada como en la reconstrucción dió el peor resultado, el sampleo unicamente para la entrada mejoró los resultados y el entrenamiento sin sampleo dió los mejores resultados. Esto se contradice con los resultados esperados y puede deberse a un problema en la implementación.

Se observó que binarizar los datos de MNIST provee mejores resultados, con lo cual se binariza los datos MNIST utilizados para entrenar.

Para el entrenamiento con sampleo en la entrada y en la reconstrucción, se utiliza un entrenamiento por minibatch de 6 muestras, una tasa de aprendizaje de 0.005 y una cantidad de 100 epochs. La Figura 4.2 muestra la evolución

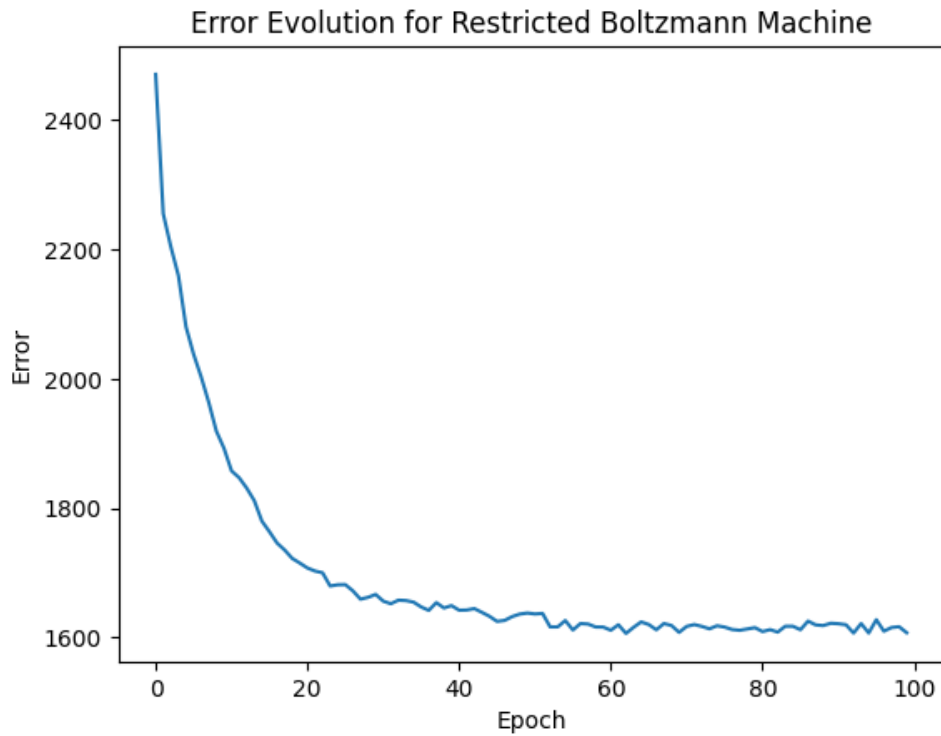


Figura 4.2: Evolución del error para la Máquina restringida de Boltzmann

Se vuelve a introducir unas imágenes MNIST y se evalúa la reconstrucción (forward-backward) de dichas imágenes en la Figura 4.3



Figura 4.3: Comparación entre las imágenes originales (arriba) y las reconstruidas con la RBM (abajo)

Se observa una mala reconstrucción de las imágenes.

Para el entrenamiento con sampleo en la entrada y sin en la reconstrucción, se utiliza un entrenamiento por minibatch de 6 muestras, una tasa de aprendizaje de 0.005 y una cantidad de 100 epochs. La Figura 4.4 muestra la evolución

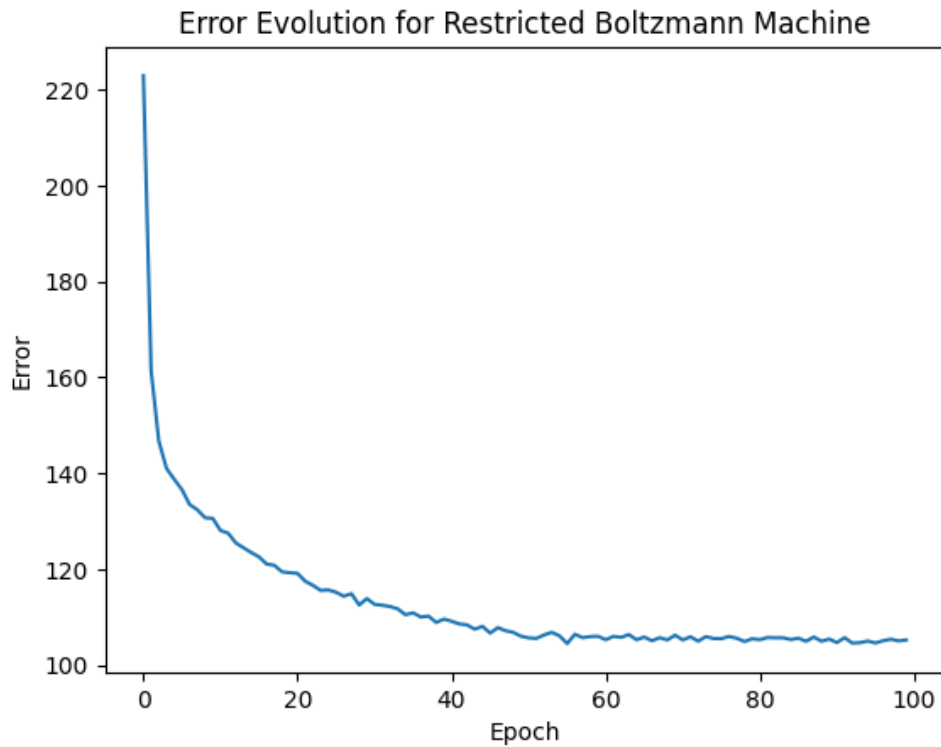


Figura 4.4: Evolución del error para la Máquina restringida de Boltzmann

Se vuelve a introducir unas imágenes MNIST y se evalúa la reconstrucción (forward-backward) de dichas imágenes en la Figura 4.5

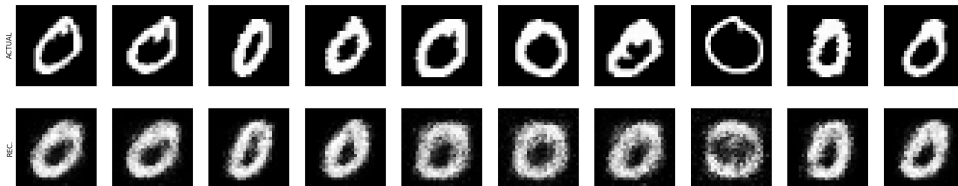


Figura 4.5: Comparación entre las imágenes originales (arriba) y las reconstruidas con la RBM (abajo)

Se observa una mejor reconstrucción de las imágenes.

Para el entrenamiento con sampleo en la entrada y en la reconstrucción, se utiliza un entrenamiento por minibatch de 4 muestras, una tasa de aprendizaje de 0.004 y una cantidad de 100 epochs. La Figura 4.6 muestra la evolución



Figura 4.6: Evolución del error para la Máquina restringida de Boltzmann

Se vuelve a introducir unas imágenes MNIST y se evalúa la reconstrucción (forward-backward) de dichas imágenes en la Figura 4.7



Figura 4.7: Comparación entre las imágenes originales (arriba) y las reconstruidas con la RBM (abajo)

Se observa la mejor reconstrucción de las imágenes. En la mayoría de los casos, la Máquina restringida de Boltzmann es sensible a la inicialización y el learning rate, resultando a veces en pesos muy grandes durante el entrenamiento.

5. Red convolucional

En esta sección, se diseña una red convolucional del tamaño mas chico posible con el fin de clasificar las imágenes de la base de datos MNIST. La base de datos MNIST consta de imágenes de 784 pixeles (28×28) obtenidas a partir de tipografías de postales. La red convolucional consta de una arquitectura similar a la que se muestra en la Figura 5.1.

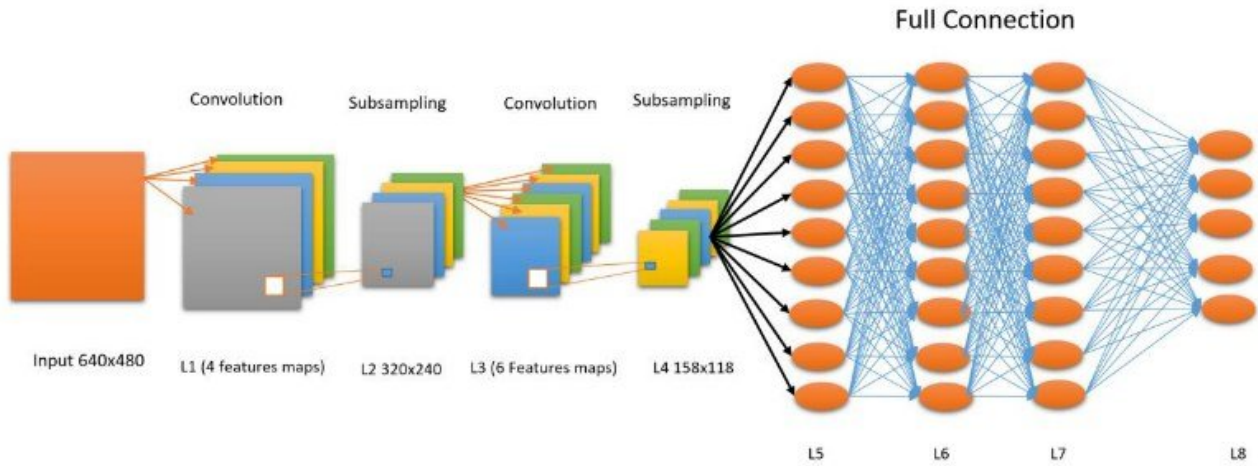


Figura 5.1: Esquema de la red neuronal convolucional

La arquitectura mínima que logra clasificar las imágenes MNIST con una precisión mayor o igual a 90 % consiste de una capa de 1 kernel de tamaño 2×2 , seguido por un max pooling (subsampling) de 3×3 , seguido por una segunda capa convolucional de 3 kernels de 2×2 , un segundo maxpooling de 2×2 y finalmente una capa lineal de 48 neuronas a 10.

En total, se tiene la siguiente cantidad de pesos a aprender:

$$\#Pesos = (2 \times 2 + 1) \times 1 + (2 \times 2 + 1) \times 3 + (48 \times 10) + 48 + 10 = 558 \quad (5.1)$$

El primer termino corresponde la dimensión del kernel mas el bias del kernel, los maxpoolings no tienen pesos, el segundo termino corresponde a la segunda capa convolucional de mismo tamaño de kernel pero 3 kernels. Los últimos términos corresponden a los pesos de la interconexión de neuronas y los pesos de los biases.

Para entrenar la red convolucional, se usa el optimizador Adam con una tasa de aprendizaje de 0.001, un minibatch size de 100 y una cantidad de 100 epochs. Para la función de perdida se usa la Cross Entropy Loss expresada de la siguiente forma

$$CEL = H(P(x)|\hat{P}(x)) = - \sum_x P(x) \log(\hat{P}(x)) \quad (5.2)$$

donde $\hat{P}(x)$ es la probabilidad categórica estimada por la red para el dato x y $P(x)$ es la verdadera probabilidad categórica para el dato x . $\hat{P}(x)$ Se estima con una salida sigmoidea

$$softmax(x, i) = \frac{e^{x(i)}}{\sum_j e^{x(j)}} \quad (5.3)$$

El error durante el entrenamiento se ve en la Figura 5.2.

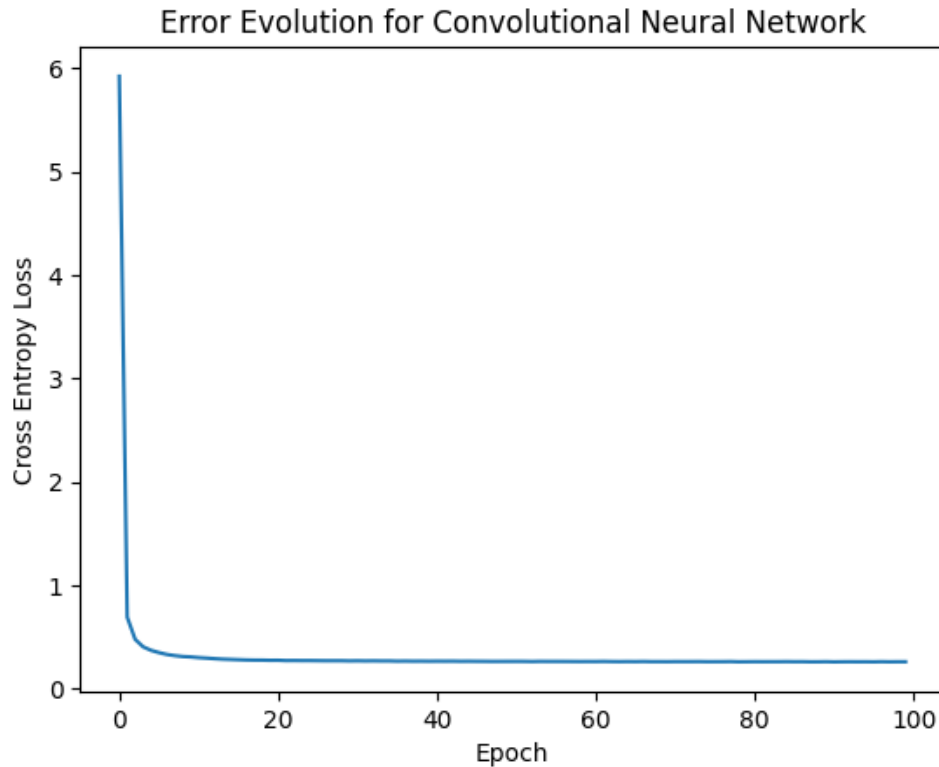


Figura 5.2: Evolución del error para la red convolucional, imagenes MNIST

La precisión alcanzada con el dataset de testeo es de 91,68 %. El calculo de la precisión es

$$Precision = \frac{\#Muestras - \#Errores}{\#Muestras} \quad (5.4)$$

Algunos ejemplos de predicciones de la red neuronal se muestran en la Figura

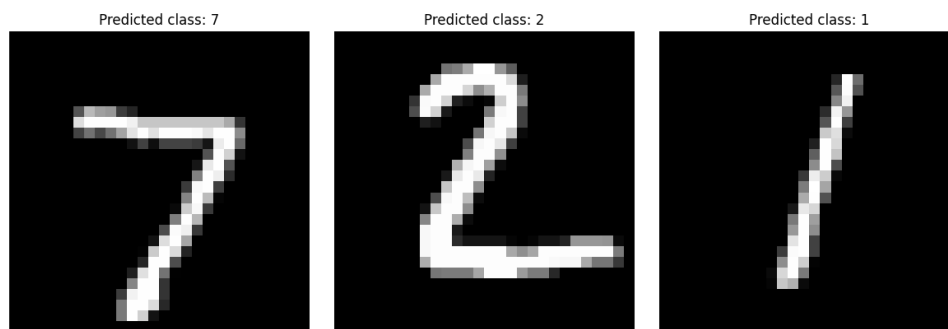


Figura 5.3: Predicciones de la red convolucional

5.1. Comparación con perceptrón multicapa

Ahora, se entrena una perceptrón multicapa que alcance una precisión de al menos 90 %. El perceptrón multicapa consta de la capa de entrada de 784 neuronas, la capa oculta de 300 y una capa de salida de 10 neuronas. La cantidad de pesos a entrenar es

$$\#Pesos = 784 \times 300 + 300 \times 10 + 784 + 300 + 10 = 239294 \quad (5.5)$$

Este numero de parámetros entrenables es significativamente mayor a la cantidad de parámetros entrenables de la red convolucional, mostrando la eficiencia de la red convolucional.

Se entrena el perceptrón multicapa con el dataset MNIST, el optimizador Adam, una tasa de aprendizaje de 0.0003 y la función de perdida Cross Entropy. El minibatchsize es 100 y el numero de epochs 100. La Figura 5.4 muestra la evolución del error durante el entrenamiento.

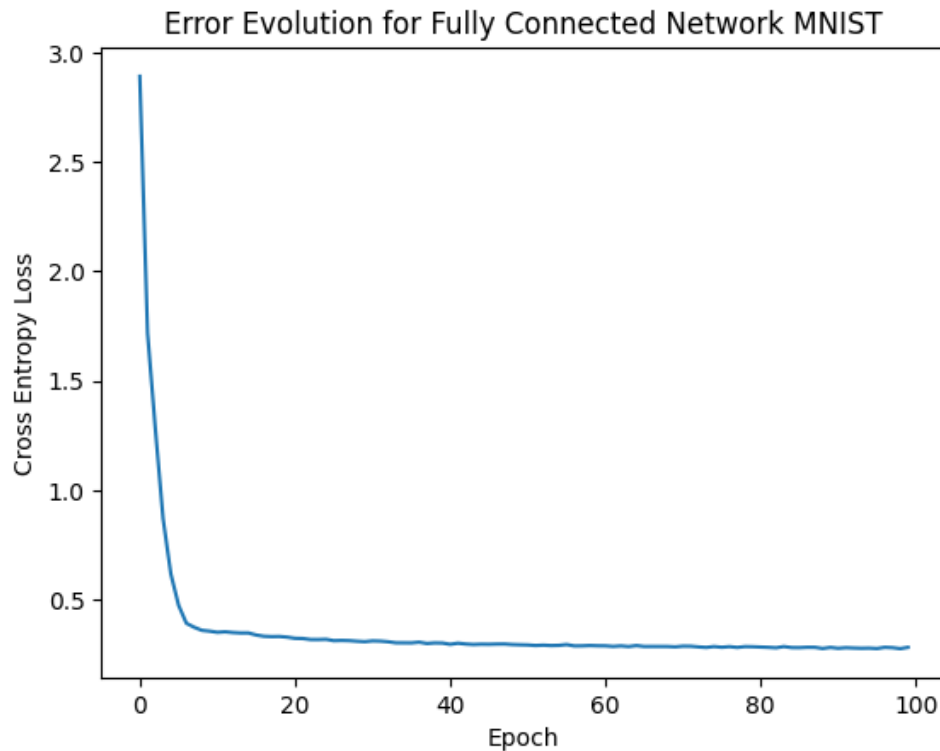


Figura 5.4: Evolución del error para el perceptrón multicapa, imágenes MNIST

La precisión del perceptrón multicapa para el dataset MNIST de testeo es de 91,18 %, similar a la red convolucional que constaba de muchos menos parámetros.

6. Autoencoder

Para el autoencoder se utiliza la arquitectura mostrada en la Figura 6.1

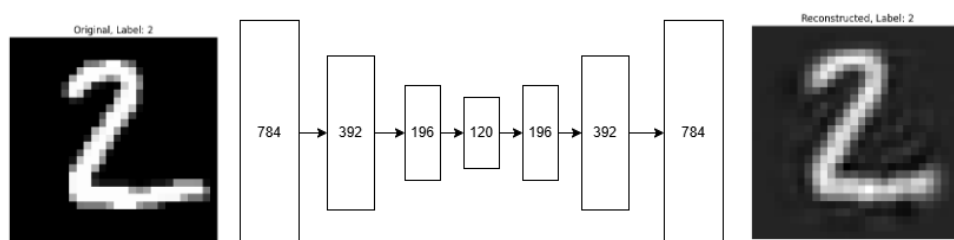


Figura 6.1: Arquitectura elegida para el autoencoder

El autoencoder es entrenado con el optimizador Adam con una tasa de aprendizaje de

$\eta = 0,0005$. La función de error es el error cuadrático medio. Se usa un minibatch de 100 muestras y una cantidad de epochs de 10. La Figura 6.2 muestra la evolución del error durante el entrenamiento con las imágenes MNIST.

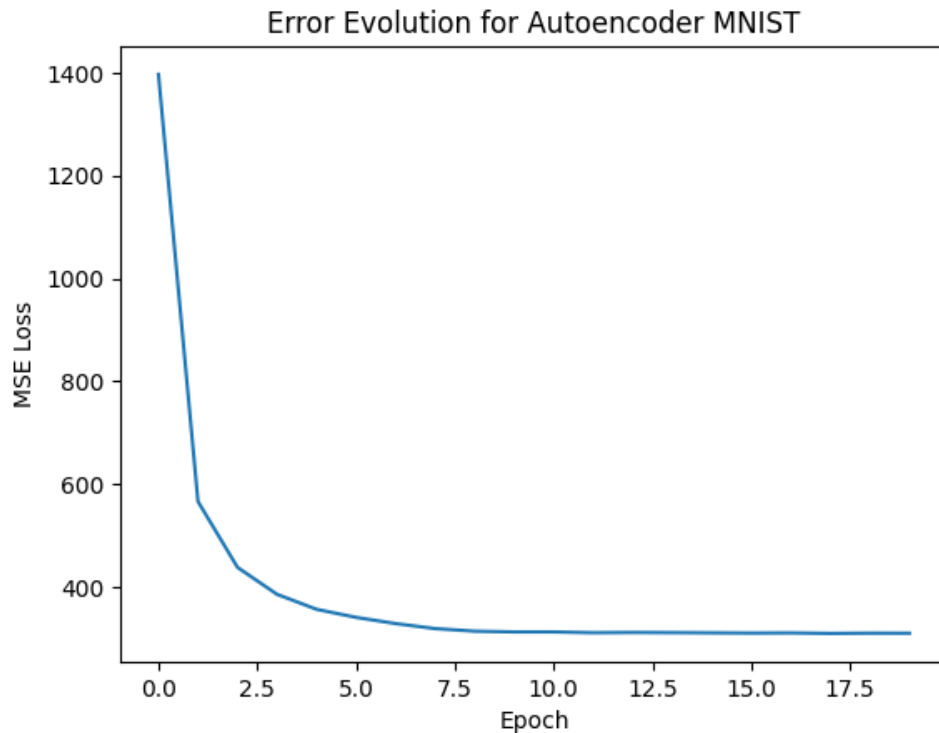


Figura 6.2: Evolución del error para el autoencoder

Para visualizar la calidad de la reconstrucción de la imagen MNIST, la Figura 6.3 muestra las imágenes originales y las reconstrucciones.

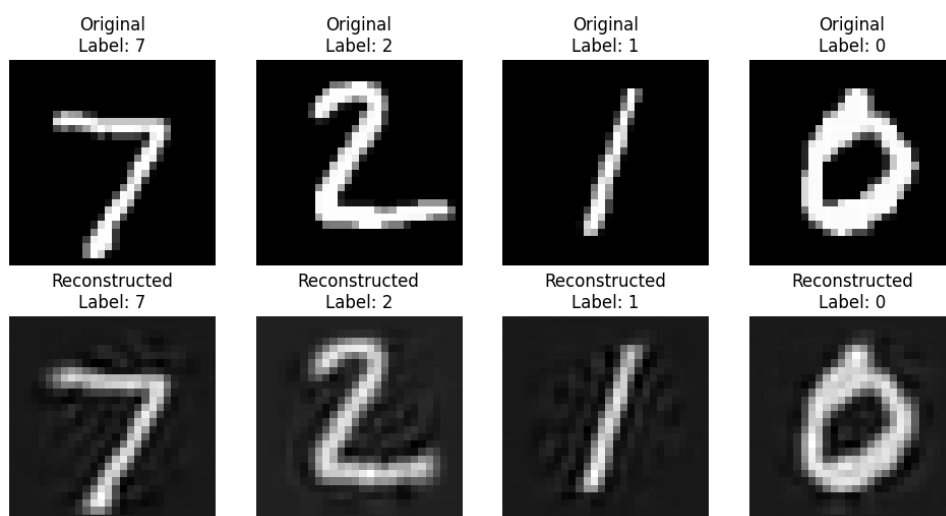


Figura 6.3: Imágenes reconstruidas con el autoencoder

Posteriormente, se usa solamente el espacio latente del autoencoder para clasificar las imágenes MNIST. El espacio latente es de dimensión 120. Se entrena ahora un perceptrón multicapa

que toma el espacio latente del autoencoder como entrada y clasifica las imágenes. El perceptrón consta de 120 neuronas de entrada 10 neuronas de salida. Nuevamente, se usa el optimizador Adam, una tasa de aprendizaje de $\eta = 0,0005$, un tamaño de minibatch de 50 y un numero de epochs de 10. El error es el Cross Entropy Loss. La Figura 6.4 muestra la evolución del error para esta etapa de clasificación que usa el espacio latente del autoencoder.

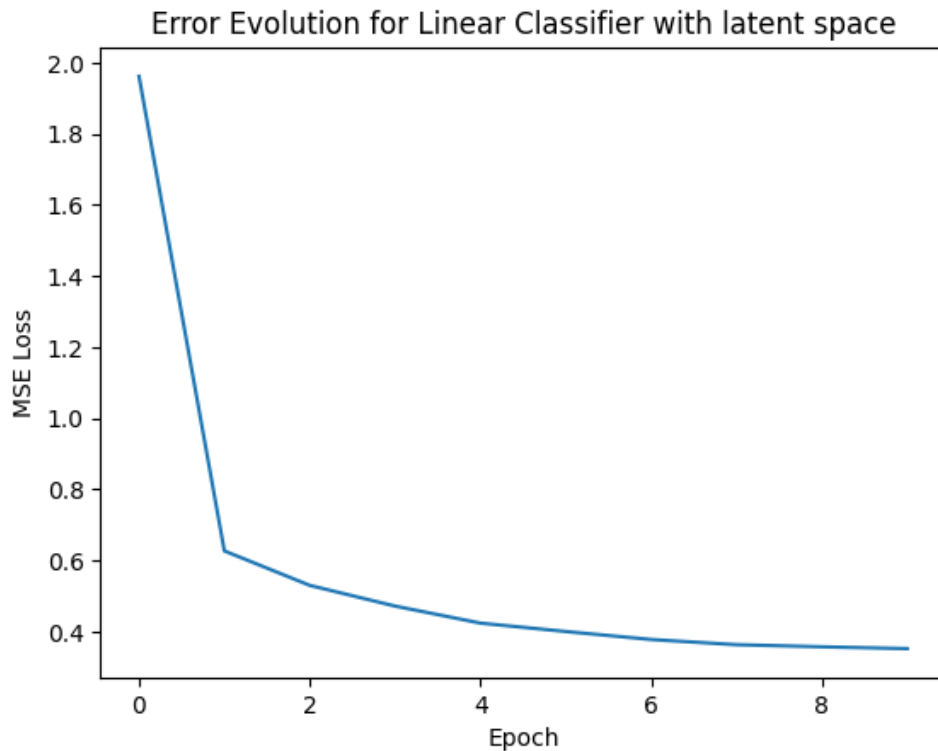


Figura 6.4: Evolución del error con clasificador que usa el espacio latente del autoencoder

La precisión alcanzada con esta red neuronal es de 90,78 %. El tiempo de entrenamiento es de $t = 13s$.

Por otro lado, si tratamos de usar el mismo clasificador sin el espacio latente, se tiene que la capa de entrada debe modificarse a 784 neuronas y la de salida sigue siendo 10 neuronas. Los parámetros de aprendizaje son los mismos que se mencionaron para el clasificador con espacio latente. La Figura 6.5 muestra la evolución del error cuadrático medio con las epochs.

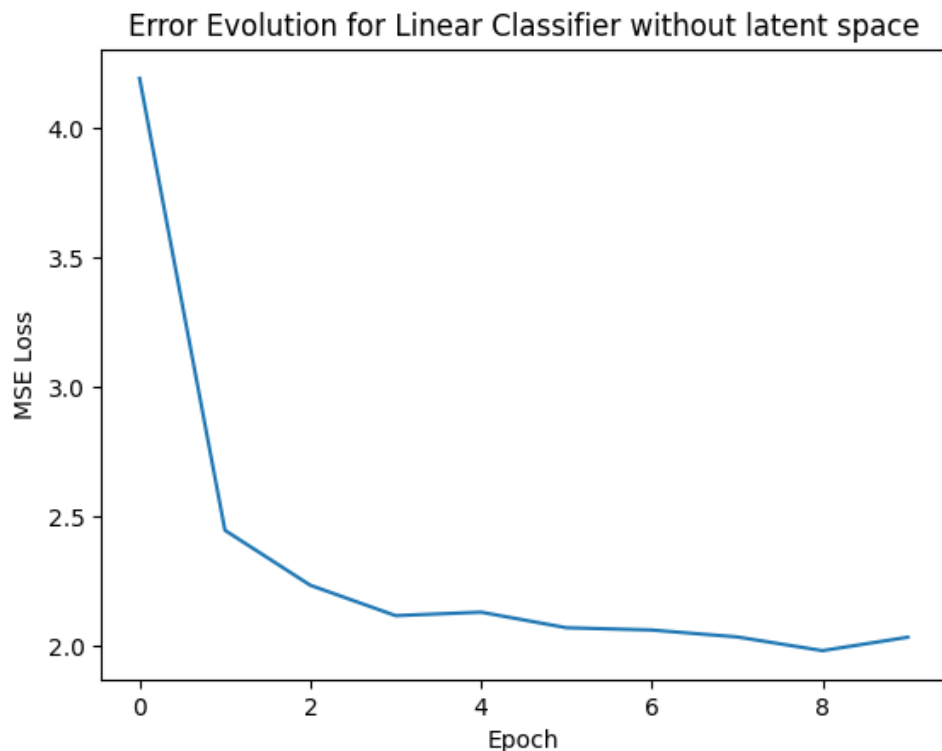


Figura 6.5: Evolución del error para el clasificador sin espacio latente

A comparación con el clasificador que usa el espacio latente, este clasificador no logra un MSE tan bajo (tiene un error 5 veces mas grande). Adicionalmente, el tiempo de entrenamiento es mayor, tomando 16.4 segundos entrenar (mayor que con espacio latente). Sorprendentemente, la precisión es de 88,4%, ligeramente menor. Sin embargo, esto se puede deber a 2 cosas: primero, a que no se sabe cual es la probabilidad que le asigno a cada clase (no se analizó, solo se seleccionó el de probabilidad mas alta, ej: 30,29,10,31 ->clase 4), segundo, puede deberse a que la base de datos MNIST utilizada carece de parámetros críticos que podrían marcar la diferencia entre los dos métodos (a mayor complejidad, el clasificador con autoencoder debería ser notablemente mejor). Por lo tanto, para evidenciar mejor la efectividad de utilizar espacio latente, debería evaluarse la correcta asignación de clase con un limite mínimo de probabilidad y debería trabajarse con datasets mas complejos.

7. Conclusión

En este trabajo práctico se pudo analizar en detalle el funcionamiento y el aprendizaje del perceptron simple y el perceptrón multicapa. Por un lado para el perceptrón simple se estudió la capacidad y por el otro para el perceptrón multicapa se implementó la regla de aprendizaje de error backpropagation mediante el gradiente descendiente. La implementación abarco problemas de clasificación y de regresión, reforzando los conceptos de learning rate y batchsize en la búsqueda de la convergencia de las redes neuronales. Por otro lado, se trabajó de cerca con la base de datos MNIST, implementando una máquina restringida de Boltzmann, entrenando una red neuronal convolucional y un autoencoder. En todos los casos se visualizaron las arquitecturas de las redes neuronales, sus métodos de aprendizaje y se estudiaron sus ventajas y desventajas. Este trabajo fue una buena forma de consolidar los conceptos aprendidos en clase a través de un método practico.

8. Apéndice

El código puede encontrarse en el siguiente link: <https://colab.research.google.com/drive/1WjtQstrvXOP6fVwWE7o5tRxczwyD-HFD?usp=sharing>